



Pizza Express

Sistema de Gestión de Pedidos e Inventario

Documentación técnica

Proyecto de Ingeniería de Software
Católica del Norte Fundación Universitaria



- **Proyecto:** Pizza Express — Sistema de Gestión de Pedidos e Inventario
 - **Versión:** 1.0 (MVP)
 - **Dirigido a:** el equipo de desarrollo y a quien tenga que mantener o ampliar el sistema
-

1. Descripción general

Pizza Express es una aplicación web que digitaliza la toma de pedidos y el control de inventario de una pizzería. Su particularidad técnica es que **el inventario se lleva a nivel de ingrediente, pero la venta se hace a nivel de pizza**: cada pizza tiene una receta, y al venderla el sistema traduce esa venta en consumo de insumos y lo descuenta en el momento.

Aspecto	Tecnología
Lenguaje	Python 3.10 o superior
Framework web	Flask 3
Plantillas	Jinja2 (incluido con Flask)
Base de datos	SQLite mediante el módulo <code>sqlite3</code> de la biblioteca estándar
Interfaz	HTML y CSS escritos a mano, sin JavaScript ni frameworks de estilo
Pruebas	<code>unittest</code> de la biblioteca estándar

La única dependencia externa del proyecto es **Flask**.

2. Arquitectura

La aplicación está organizada en tres capas. La regla que se respeta en todo el código es que **las dependencias van siempre hacia abajo**: las vistas conocen los servicios, los servicios conocen los modelos y la base de datos, pero nunca al revés.

PRESENTACIÓN `app/vistas/ + app/plantillas/`
Blueprints de Flask y plantillas Jinja2.
Leen el formulario, llaman a un servicio y renderizan.

LÓGICA DE NEGOCIO `app/servicios/`
Reglas del negocio. No importan Flask: reciben una conexión sqlite3, por eso se pueden probar sin servidor.

DATOS `app/modelos/ + app/basedatos.py`
Estructuras de datos (dataclasses) y acceso a SQLite.

Por qué esta separación. Permite probar toda la lógica crítica —descuento de inventario, bloqueo de venta, ciclo de estados— con `unittest` y una base en memoria, sin levantar el servidor web. Las 18 pruebas del proyecto se apoyan en eso.

Flujo de una petición

Ejemplo: el cajero guarda un pedido nuevo.

1. El navegador envía `POST /pedidos/nuevo`.
2. El decorador `requiere_rol("administrador", "cajero")` comprueba la sesión y el rol.
3. La vista `app/vistas/pedidos.py::nuevo` lee el formulario y arma la lista de líneas.
4. Llama a `app/servicios/pedidos.py::crear_pedido`, que valida, comprueba el inventario, inserta el pedido y descuenta los insumos **en una sola transacción**.
5. Si el servicio lanza `ErrorDeNegocio`, la vista lo captura, muestra el mensaje y vuelve a pintar el formulario. Si todo va bien, redirige al detalle del pedido.

3. Estructura del proyecto

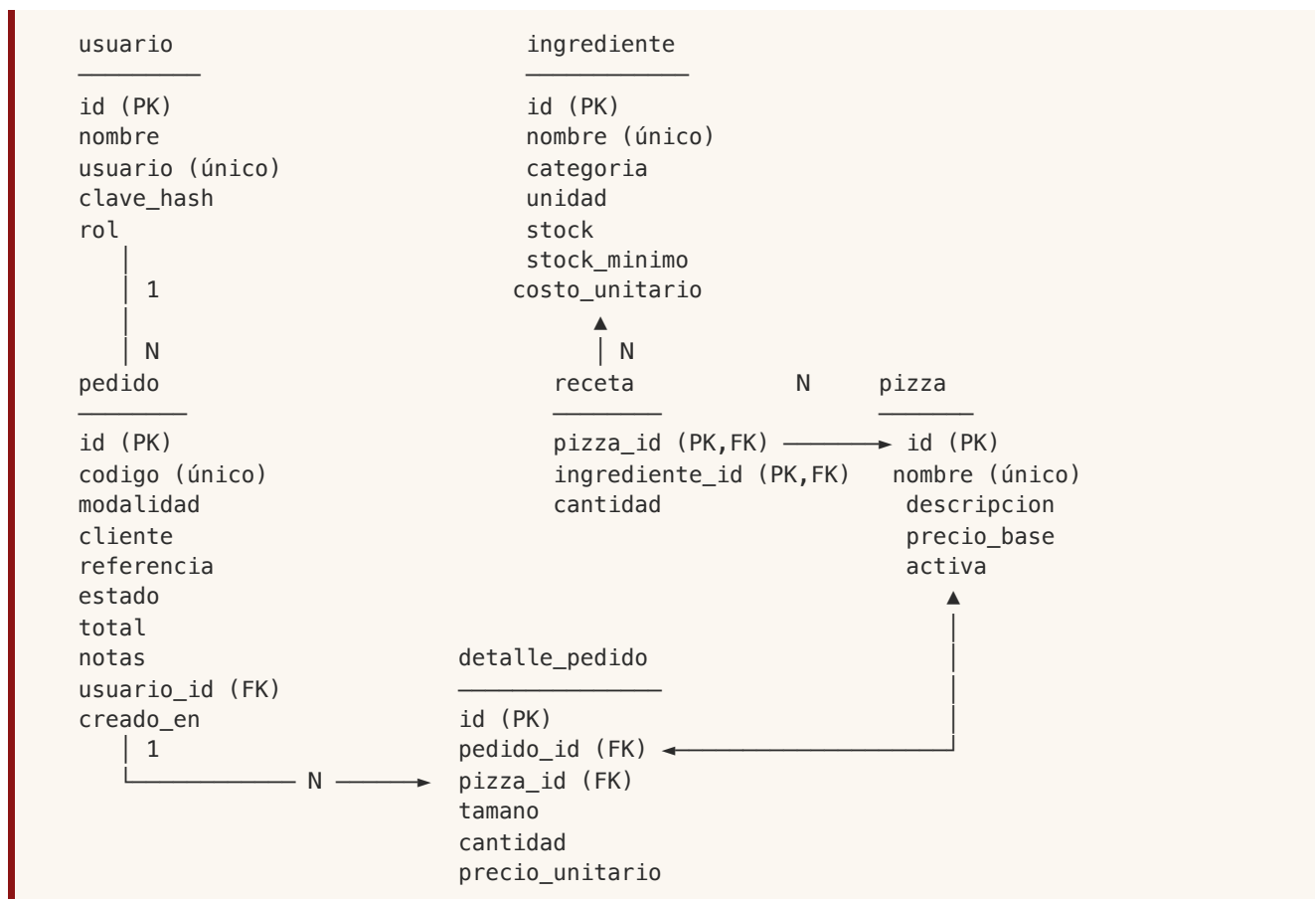
```
Ing-Soft-Pizzeria-Python/
├── ejecutar.py           # Punto de entrada
├── requirements.txt      # Flask
├── assets/              # Carpeta estática de Flask, servida en /assets
│   ├── logo.png        # Logo corporativo
│   └── estilos.css      # Hoja de estilos
├── datos/pizzaexpress.db # Base de datos (se genera sola)
├── docs/                # Documentación del proyecto
├── pruebas/test_servicios.py # Pruebas automáticas
├── app/
│   ├── __init__.py      # crear_app(): fábrica de la aplicación
│   ├── configuracion.py # Constantes de negocio y de marca
│   ├── basedatos.py     # Conexión, esquema DDL y datos de ejemplo
│   ├── utilidades.py    # Formatos y control de acceso
│   ├── modelos/         # Ingrediente, Pizza, Pedido, Usuario
│   ├── servicios/       # inventario, menu, pedidos, reportes, autenticacion
│   ├── vistas/          # Un blueprint por sección
│   └── plantillas/      # Plantillas Jinja2
```

Responsabilidad de cada módulo

Módulo	Responsabilidad
app/configuracion.py	Nombre y lema de la marca, rutas, factores de tamaño, modalidades, estados y transiciones permitidas, roles. Es el único sitio donde se tocan estos valores
app/basedatos.py	Esquema SQL, apertura de conexiones y siembra de datos de ejemplo
app/utilidades.py	ErrorDeNegocio , formatos de moneda/cantidad/fecha y los decoradores requiere_sesion y requiere_rol
app/modelos/	Dataclasses que representan las filas y calculan propiedades derivadas (esta_agotado , subtotal , siguientes_estados)
app/servicios/inventario.py	Consumo por receta, disponibilidad, descuento y devolución, alertas y reabastecimiento
app/servicios/menu.py	Catálogo, recetas, precios por tamaño y disponibilidad de cada pizza
app/servicios/pedidos.py	Creación de pedidos, códigos, consultas y ciclo de estados
app/servicios/reportes.py	Agregados de ventas por día, modalidad, tamaño y ranking de pizzas
app/vistas/	Blueprints de Flask; solo orquestan, no contienen reglas de negocio

4. Modelo de datos

Diagrama entidad-relación



La tabla **receta** es la pieza central del diseño: es la que permite vender pizzas y descontar ingredientes. Sin ella el sistema no podría cumplir el requerimiento del descuento automático.

Diccionario de datos

usuario

Campo	Tipo	Descripción
id	INTEGER PK	Identificador
nombre	TEXT	Nombre que se muestra en pantalla
usuario	TEXT único	Nombre de acceso
clave_hash	TEXT	SHA-256 de la contraseña
rol	TEXT	administrador , cajero o pizzero

ingrediente

Campo	Tipo	Descripción
id	INTEGER PK	Identificador
nombre	TEXT único	Nombre del insumo
categoria	TEXT	Agrupación para filtrar (Lácteos, Carnes, Verduras...)
unidad	TEXT	Unidad de medida: kg, L, unidad
stock	REAL	Cantidad disponible ahora mismo
stock_minimo	REAL	Umbral que dispara la alerta
costo_unitario	REAL	Costo de compra por unidad

pizza

Campo	Tipo	Descripción
id	INTEGER PK	Identificador
nombre	TEXT único	Nombre comercial
descripcion	TEXT	Texto que ve el cliente
precio_base	REAL	Precio de la pizza mediana
activa	INTEGER	1 si está en el menú, 0 si se retiró

receta

Campo	Tipo	Descripción
<code>pizza_id</code>	INTEGER PK, FK	Pizza a la que pertenece
<code>ingrediente_id</code>	INTEGER PK, FK	Insumo que consume
<code>cantidad</code>	REAL	Cantidad para una pizza mediana

pedido

Campo	Tipo	Descripción
<code>id</code>	INTEGER PK	Identificador
<code>codigo</code>	TEXT único	Código legible: PED-0001
<code>modalidad</code>	TEXT	<code>mesa</code> , <code>llevar</code> o <code>domicilio</code>
<code>cliente</code>	TEXT	Nombre del cliente
<code>referencia</code>	TEXT	Número de mesa, teléfono o dirección, según la modalidad
<code>estado</code>	TEXT	<code>pendiente</code> , <code>en_preparacion</code> , <code>lista</code> , <code>entregado</code> o <code>cancelado</code>
<code>total</code>	REAL	Total calculado al crear el pedido
<code>notas</code>	TEXT	Indicaciones para la cocina
<code>usuario_id</code>	INTEGER FK	Quién tomó el pedido
<code>creado_en</code>	TEXT	Fecha y hora locales

detalle_pedido

Campo	Tipo	Descripción
<code>id</code>	INTEGER PK	Identificador
<code>pedido_id</code>	INTEGER FK	Pedido al que pertenece (borrado en cascada)
<code>pizza_id</code>	INTEGER FK	Pizza vendida
<code>tamano</code>	TEXT	<code>personal</code> , <code>mediana</code> o <code>familiar</code>
<code>cantidad</code>	INTEGER	Número de pizzas de esta línea
<code>precio_unitario</code>	REAL	Precio congelado en el momento de la venta

Se guarda `precio_unitario` en lugar de leer el precio actual de la pizza para que un cambio de precios no altere el importe de los pedidos ya cerrados ni los reportes históricos.

Integridad

Las claves foráneas están activas (`PRAGMA foreign_keys = ON`). El borrado de un pedido arrastra su detalle (`ON DELETE CASCADE`), mientras que un ingrediente o una pizza usados en una receta o en un pedido no se pueden borrar (`ON DELETE RESTRICT`), para no dejar historial inconsistente.

5. Reglas de negocio y algoritmos

5.1 Tamaños

Definidos en `app/configuracion.py`:

Tamaño	Factor
Personal	0,6
Mediana	1,0
Familiar	1,6

El factor multiplica **a la vez** el precio de venta y las cantidades de la receta. Así, una pizza familiar cuesta 1,6 veces la mediana y consume 1,6 veces sus ingredientes. El precio se redondea a centenas en

`Pizza.precio_por_tamano` (`app/modelos/pizza.py`).

5.2 Cálculo del consumo — `calcular_consumo`

En `app/servicios/inventario.py`. Recibe las líneas del pedido y devuelve un diccionario

`{ingrediente_id: cantidad_total}`:

```
para cada línea del pedido:
    factor = factor del tamaño
    para cada ingrediente de la receta de esa pizza:
        consumo[ingrediente] += cantidad_receta × cantidad_pedida × factor
```

La acumulación es importante: si el pedido lleva una Margarita y una Pepperoni, ambas usan harina, y hay que sumar las dos cantidades antes de comprobar si hay suficiente. Comprobarlas por separado dejaría pasar pedidos que en conjunto no se pueden preparar.

5.3 Bloqueo de venta — `verificar_disponibilidad`

Recorre el consumo calculado y compara con el stock. Devuelve `(hay_stock, faltantes)`, donde `faltantes` son mensajes listos para mostrar:

Falta «Queso azul»: se necesitan 0.08 kg y solo hay 0 kg.

`crear_pedido` llama a esta función **antes** de escribir nada. Si falla, lanza `ErrorDeNegocio` y no se inserta el pedido ni se toca el inventario.

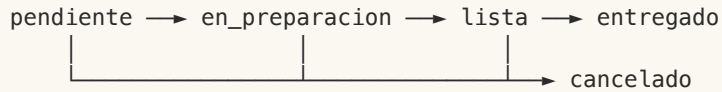
5.4 Transacción de creación del pedido

En `app/servicios/pedidos.py::crear_pedido`, el orden es:

1. Validar modalidad, cliente y referencia obligatoria.
2. Validar las líneas y congelar el precio de cada una.

3. **Verificar disponibilidad** (si falla, se aborta aquí).
4. Insertar la cabecera y el detalle.
5. Descontar el inventario.
6. `commit`. Si algo falla en los pasos 4-5, se hace `rollback` y no queda nada a medias.

5.5 Ciclo de estados



Las transiciones válidas están declaradas en `TRANSICIONES_PERMITIDAS` (`app/configuracion.py`), y `cambiar_estado` las respeta: no se puede saltar un estado ni cancelar un pedido ya entregado. Al pasar a `cancelado` se llama a `devolver_receta`, que reintegra al inventario exactamente lo que se había descontado.

5.6 Alertas

Un ingrediente entra en alerta cuando `stock <= stock_minimo`; si además `stock <= 0` se clasifica como *agotado*. La consulta está en `obtener_alertas` y el contador del menú lo alimenta `contar_alertas`, que se inyecta en todas las plantillas desde el `context_processor` de `app/__init__.py`.

5.7 Reportes

Todas las consultas de `app/servicios/reportes.py` excluyen los pedidos cancelados con la condición `p.estado <> 'cancelado'`, de modo que una cancelación nunca infla las ventas.

6. Mapa de rutas

Ruta	Método	Blueprint	Roles permitidos
/login	GET, POST	autenticacion	Público
/logout	GET	autenticacion	Con sesión
/	GET	panel	Todos
/pedidos/	GET	pedidos	administrador, cajero
/pedidos/nuevo	GET, POST	pedidos	administrador, cajero
/pedidos/<id>	GET	pedidos	administrador, cajero
/pedidos/<id>/estado	POST	pedidos	administrador, cajero
/cocina/	GET	cocina	administrador, pizzero
/cocina/<id>/estado	POST	cocina	administrador, pizzero
/inventario/	GET	inventario	administrador, cajero
/inventario/alertas	GET	inventario	Todos
/inventario/nuevo	GET, POST	inventario	administrador
/inventario/<id>/editar	GET, POST	inventario	administrador
/inventario/<id>/reabastecer	POST	inventario	administrador
/menu/	GET	menu	Todos
/menu/<id>/precio	POST	menu	administrador
/menu/<id>/disponibilidad	POST	menu	administrador
/reportes/	GET	reportes	administrador
/assets/<archivo>	GET	(estático)	Público

7. Seguridad

Lo que sí implementa el prototipo:

- Las contraseñas no se guardan en texto plano: se almacena su **hash SHA-256** (`app/basedatos.py::cifrar_clave`).
- La sesión viaja en una **cookie firmada** por Flask con `SECRET_KEY`.
- Todas las vistas están protegidas con `requiere_sesion` o `requiere_rol` (`app/utilidades.py`); no basta con conocer la URL para entrar.
- Todas las consultas usan **parámetros** (`?`), nunca concatenación de cadenas, lo que evita inyección SQL.
- Jinja2 escapa el HTML por defecto, lo que evita XSS en los datos que escribe el usuario.

Limitaciones conocidas, por tratarse de un prototipo académico:

- SHA-256 sin *salt* ni derivación lenta: en producción debería usarse `bcrypt` o `argon2`.
- La `SECRET_KEY` está fija en el código; debería venir de una variable de entorno.
- No hay protección CSRF en los formularios.
- Se usa el servidor de desarrollo de Flask, que no es apto para producción.
- Los formularios de cambio de estado no llevan token, por lo que el sistema está pensado para una red local de confianza (la de la pizzería).

8. Decisiones técnicas

Decisión	Motivo	Alternativa descartada
Flask en lugar de Django	Menos configuración generada, una sola dependencia y código más fácil de explicar en la sustentación	Django, propuesto en la presentación; aporta ORM y panel de administración que este alcance no necesita
SQLite con <code>sqlite3</code>	No requiere instalar ni administrar un servidor de base de datos; el proyecto funciona al descomprimirlo	PostgreSQL o MySQL, innecesarios para un prototipo de un solo puesto
SQL escrito a mano	Deja el modelo de datos a la vista, que es parte de lo que se evalúa	Un ORM ocultaría precisamente lo que hay que mostrar
Sin JavaScript	La aplicación funciona sin conexión y toda la validación importante queda en el servidor	Una interfaz de una sola página exigiría una capa extra de complejidad
Receta como tabla propia	Es lo que permite vender pizzas descontando ingredientes	Guardar la receta como texto impediría calcular el consumo
Guardar <code>precio_unitario</code> en el detalle	Los pedidos históricos no cambian si se actualizan los precios	Leer el precio actual falsearía los reportes

Nota sobre los decimales

El stock se guarda como `REAL` (coma flotante), así que pueden aparecer valores como `23.619999999999997` en consultas directas a la base de datos. Es el comportamiento normal de la aritmética de punto flotante y no afecta a la operación, porque la interfaz formatea las cantidades con `formato_cantidad` (`app/utilidades.py`). Si el sistema pasara a producción con manejo contable estricto, convendría guardar las cantidades como enteros en la unidad mínima (gramos, mililitros) o usar `Decimal`.

9. Instalación y ejecución

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python ejecutar.py                 # http://127.0.0.1:5000
```

La base de datos se crea y se siembra sola la primera vez. Para reiniciar los datos basta con borrar `datos/pizzaexpress.db`.

Pruebas:

```
python -m unittest discover -s pruebas -v
```

10. Guía de mantenimiento

Añadir una pizza al menú. Insertar la fila en `pizza` y sus líneas en `receta`. Si es para los datos de ejemplo, basta con añadir la tupla a `PIZZAS_INICIALES` en `app/basedatos.py` —incluyendo el diccionario de la receta— y volver a generar la base.

Añadir un ingrediente. Desde la aplicación con el rol de administrador (`/inventario/nuevo`), o añadiéndolo a `INGREDIENTES_INICIALES`.

Cambiar los tamaños o sus factores. Solo hay que tocar `TAMANOS` en `app/configuracion.py`; el precio y el consumo se recalculan solos en toda la aplicación.

Añadir un estado al pedido. Declararlo en `ESTADOS` y definir sus transiciones en `TRANSICIONES_PERMITIDAS` (`app/configuracion.py`). Las plantillas de cocina y de detalle leen esas constantes, así que se adaptan automáticamente.

Añadir un rol. Registrarlo en `ROLES` (`app/configuracion.py`), incluirlo en los decoradores `requiere_rol` de las vistas que deba ver y añadir el enlace correspondiente en la navegación de `app/plantillas/base.html`.